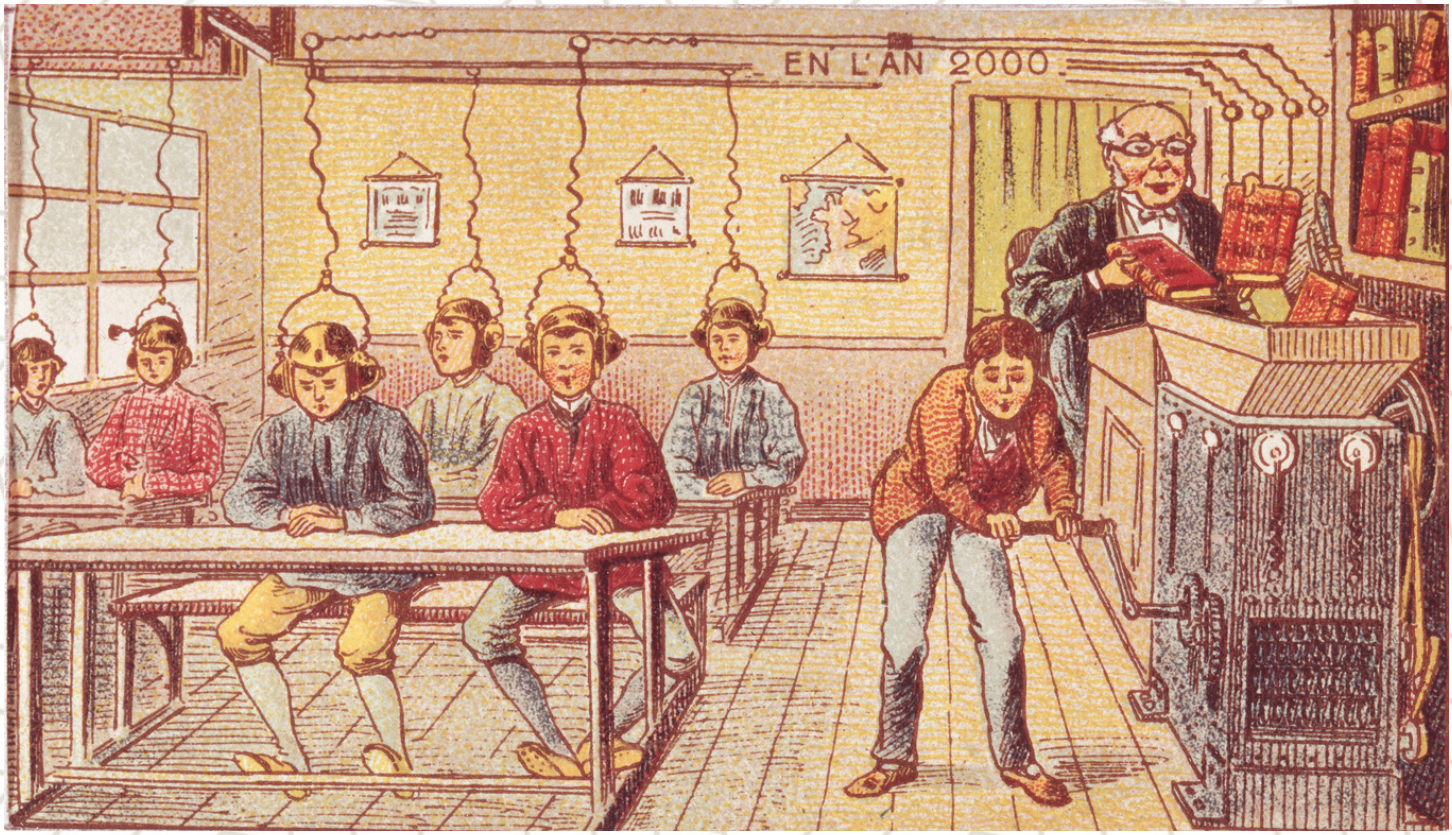
MTDI24

Consegne delle attività di:
Pierri Salvatore

Matricola:
1043293



CONSEGNA 1

Nel saggio “Informatica: la terza rivoluzione dei rapporti di potere”, condivido pienamente le riflessioni dell'autore, in particolare un passaggio finale: “Ci sono però due caratteristiche distintive di queste ‘macchine cognitive’ che le rendono diverse dal modo di operare dell’essere umano: la flessibilità e l’adattabilità.” Infatti, poco prima, l'autore spiega come le macchine superino le capacità umane in diverse situazioni. Tuttavia, ritengo che senza un input o una rete neurale in grado di cercare informazioni online, le macchine non possano generare dati autonomamente, né scoprire nuove abilità da zero o esplorare contesti senza l'assistenza di un intervento umano.

Nel secondo testo, “Informatica e competenze digitali: cosa insegnare?”, mi sento molto in sintonia con quanto afferma l'autore, in particolare riguardo a questa frase: "Nel mondo della scuola, in particolare, c’è confusione tra i due aggettivi ‘informatiche’ e ‘digitali’ e i relativi sostantivi. Ma non è affatto colpa dei docenti, né cercare chi sia il colpevole ha una qualche utilità." Durante i miei primi due anni di scuola superiore, ho seguito un corso di informatica (generale) che in realtà si limitava a Word ed Excel, software che ogni ragazzo della nostra età ha sempre utilizzato in modo autonomo e che non rientra affatto nella definizione di informatica come disciplina accademica.

Nel terzo testo, “MANIFESTO DI VIENNA PER L’UMANESIMO DIGITALE”, condivido la frase: “L’insorgere attraverso i social media di opinioni e comportamenti estremisti, la formazione di bolle informative (filter bubbles) e di casse di risonanza (echo chambers) che promuovono verità dissonanti”. Oggi, con un minimo di attenzione, ogni parola viene percepita come verità assoluta da una parte della popolazione che non si impegna a informarsi (la maggioranza). È sorprendente come, in un'epoca in cui le informazioni sono accessibili gratuitamente e in modo massiccio, non ci sia la volontà di effettuare una ricerca critica. Al contrario, si tende a credere alla prima informazione che ci viene proposta da qualche guru, che probabilmente non ha competenze nel settore e che potrebbe essere motivato unicamente dall'idea di guadagnare una fortuna.

CONSEGNA 1

Nel terzo testo, "MANIFESTO DI VIENNA PER L'UMANESIMO DIGITALE", non condivido la frase: "Dobbiamo progettare le tecnologie in base ai valori e ai bisogni umani, piuttosto che permettere alle tecnologie di plasmare gli esseri umani". Ritengo che questo obiettivo sia irrealizzabile; come dimostrato dalla storia, le nuove tecnologie influenzano l'umanità in ogni suo aspetto, come è accaduto con il linguaggio in seguito all'avvento di internet.

A mio avviso, l'informatica fa parte del campo scientifico, poiché il termine scienza deriva dal latino "scientia", che significa "conoscenza". La parola informatica, a sua volta, nasce dalla combinazione delle parole latine e greche "informatio" e "automatos", traducibile come "informazione automatica".
È difficile non associare la conoscenza generale a un'informazione automatica, che oggi possiamo considerare globale, poiché l'informatica è presente in ogni istante della nostra vita quotidiana. Nella mia concezione di materie scientifiche rientrano tutte quelle discipline che utilizziamo inconsapevolmente senza conoscerle, come fisica, chimica, biologia, matematica e, di conseguenza, anche l'informatica.

L'INFORMATICA INOLTRE SODDISFA TUTTI I REQUISITI DELL'ARTICOLO: The Science in Computer Science.

Organized to understand, exploit, and cope with a pervasive phenomenon

L'informatica è nata con l'obiettivo specifico di comprendere, gestire e sfruttare il fenomeno dell'informazione. La disciplina si è sviluppata per affrontare in modo sistematico il trattamento dei dati e l'automazione dei processi informativi, che oggi rappresentano un fenomeno pervasivo nella società contemporanea.

CONSEGNA 1

Encompasses natural and artificial processes of the phenomenon

L'informatica, originariamente concepita per la progettazione di macchine digitali, si è progressivamente estesa alla comprensione di processi sia artificiali sia naturali. Oggi essa integra l'uso di strumenti digitali con metodologie di ricerca interdisciplinare, contribuendo allo studio di fenomeni in numerosi ambiti scientifici.

Codified structured body of knowledge

L'informatica si fonda su convenzioni formali e standard riconosciuti a livello globale, ed è caratterizzata dall'uso di una terminologia tecnica specifica che garantisce precisione e uniformità nella comunicazione scientifica.

Commitment to experimental methods for discovery and validation

L'informatica è una disciplina in continuo sviluppo, caratterizzata da una costante evoluzione favorita dalla libertà di sperimentazione. Gran parte delle nuove scoperte informatiche nasce dall'ideazione di nuovi algoritmi o dal miglioramento di software esistenti, spesso a partire da codice open source, attraverso processi di sperimentazione e innovazione progressiva.

Reproducibility of results

In informatica, non solo i risultati sono generalmente riproducibili, ma spesso vengono anche resi disponibili attraverso librerie pubbliche di facile utilizzo. Grazie ai processi di astrazione e generalizzazione, tali strumenti permettono di applicare soluzioni complesse a una vasta gamma di problemi in modo standardizzato ed efficiente.

Falsifiability of hypotheses and models

In informatica, qualsiasi modello può essere messo in discussione qualora venga individuato un controesempio che ne smentisca la validità. Questo principio riflette la natura falsificabile dei modelli informatici, elemento fondamentale per il rigore scientifico della disciplina.

Ability to make reliable predictions, some of which are surprising

Un esempio semplice è rappresentato dalle previsioni meteorologiche quotidiane, che si basano su predizioni generate da modelli informatici, elaborando dati raccolti e trattati anch'essi in formato digitale.

CONSEGNA 2

OBBIETTIVI DELLA LEZIONE

- **Obiettivi Generali:**
  - **La lezione si propone di sviluppare sia la conoscenza concettuale sia la conoscenza procedurale dell'argomento. Al termine dell'attività, gli studenti dovranno essere in grado di riconoscere la struttura di un algoritmo, riconoscere le ambiguità del linguaggio normale e, attraverso il ragionamento mentale applicato agli pseudoalgoritmi proposti, valutare autonomamente se una procedura possa essere effettivamente classificata come algoritmo.**

- **Obiettivi Cognitivi:**
  - **Comprendere il concetto di algoritmo, inteso come una sequenza ordinata di istruzioni finalizzata alla risoluzione di un problema.**
  - **Comprendere le proprietà fondamentali di un algoritmo, quali, ad esempio la presenza di un inizio e di una fine ben definiti.**

- **Obiettivi educativi**
  - **Promuovere lo sviluppo del pensiero logico e del pensiero critico.**
  - **Stimolare la collaborazione e il confronto costruttivo tra pari.**

SVOLGIMENTO & DISCUSSIONE

- **Informazioni di base**
  - **La lezione sarà strutturata come un'attività mista di lezione frontale ed esercitazione pratica, basata sull'integrazione di due strategie di active learning: Learning Cycle Instructional Model (5E) & Think-Pair-Share. L'intervento didattico si articolerà in cinque fasi, con l'obiettivo di assicurare che ogni studente, senza eccezioni, possa assimilare i contenuti proposti.**

CONSEGNA 2

- **Fase 1: Introduzione**
  - **Il docente fornirà una breve introduzione sul concetto di algoritmo, distribuendo agli studenti una definizione sintetica che verrà utilizzata come riferimento per la fase successiva.**

- **Fase 2: Lavoro individuale**
  - **Gli studenti avranno a disposizione un intervallo di tempo (10-20 minuti) per rispondere individualmente alla seguente domanda: "Questo pseudoalgoritmo può essere classificato come algoritmo?" Le risposte, corredate da motivazioni, verranno annotate sui rispettivi quaderni, così da poter essere successivamente confrontate e discusse.**

- **Fase 3: Discussione guidata**
  - **Il docente selezionerà in modo casuale 3-4 studenti (preferibilmente includendo anche alunni normalmente meno partecipativi, al fine di favorire il coinvolgimento di tutta la classe) e chiederà loro di leggere ad alta voce le proprie risposte.**
  - **Al termine delle esposizioni, il docente introdurrà formalmente il concetto di proprietà degli algoritmi, utilizzando esempi concreti tratti dagli pseudoalgoritmi analizzati (ad esempio, illustrando come nello pseudoalgoritmo delle "lasagne" sia necessario specificare ogni fase, come la preparazione del ragù, senza dare nulla per scontato).**

- **Fase 4: Lavoro di gruppo**
  - **Successivamente, gli studenti si riuniranno in gruppi omogenei, composti da membri sempre diversi rispetto a quelli precedenti. Con le nuove informazioni apprese, i gruppi modificheranno e aggiorneranno le risposte precedentemente formulate, discutendo tra loro per raggiungere una soluzione condivisa per ogni pseudoalgoritmo proposto.**

CONSEGNA 2

- **Fase 5: Presentazione e confronto finale**
  - **Nella fase conclusiva, il docente inviterà un gruppo alla volta a esporre le proprie risposte attraverso un "leader" scelto internamente dagli studenti.**
  - **Successivamente, ogni gruppo dovrà dichiarare se concorda o discorda con quanto esposto. In caso di discordanze, si procederà all'analisi dettagliata delle risposte divergenti, con l'obiettivo di giungere collettivamente alla soluzione corretta, favorendo così il consolidamento delle conoscenze.**

CONCLUSIONE

- **Il docente riassume brevemente tutti i principali concetti appresi, con un breve recap come ad esempio:**
  - **Un algoritmo è una sequenza ordinata di istruzioni che consente di risolvere un problema o portare a termine un compito.**
  - **Gli algoritmi devono possedere alcune proprietà fondamentali:**
    - **Finità: devono terminare dopo un numero finito di passi.**
    - **Precisione: ogni istruzione deve essere chiara e non ambigua.**
    - **Fattibilità: ogni operazione deve poter essere eseguita realmente.**
    - **Presenza di input e output: l’algoritmo elabora dati in ingresso per produrre risultati in uscita.**

CONSEGNA 2

Obiettivi futuri dopo la lezione introduttiva sugli algoritmi

A seguito della lezione introduttiva sul concetto di algoritmo, ritengo che gli obiettivi futuri per la classe possano articolarsi lungo diverse direttrici di approfondimento:

- **Creazione autonoma di algoritmi**
  - **Gli studenti saranno guidati a costruire sequenze di istruzioni per risolvere problemi semplici, sviluppando così la capacità di progettare algoritmi in maniera autonoma.**
- **Connessione con concetti matematici**
  - **Verranno introdotti esercizi che colleghino gli algoritmi a operazioni matematiche di base, favorendo il consolidamento del pensiero procedurale anche nell'ambito logico-matematico.**
- **Introduzione al coding visuale**
  - **Attraverso strumenti come Flowgorithm, gli alunni potranno muovere i primi passi nella rappresentazione grafica degli algoritmi, sviluppando competenze di base nella stesura di diagrammi di flusso.**
- **Apprendimento di condizioni e cicli**
  - **Successivamente, si procederà all'introduzione dei concetti fondamentali di condizioni logiche (if-then-else) e cicli (ripetizioni), elementi centrali nella costruzione di algoritmi più articolati.**

CONSEGNA 2

A titolo personale, ritengo che, successivamente a questa lezione introduttiva, sarebbe particolarmente efficace proporre agli alunni un'attività pratica su code.org/minecraft.

Questa piattaforma integra il celebre gioco Minecraft con l'insegnamento dei principi di base della programmazione e degli algoritmi, offrendo un ambiente ludico ed educativo altamente motivante.

Ho avuto modo di scoprire e utilizzare autonomamente questo strumento durante il mio percorso scolastico nella scuola primaria, riscontrando come esso favorisca l'apprendimento dei concetti algoritmici attraverso il gioco e la sperimentazione attiva.

TEMI INFORAMTICI AFFRONTATI NELL'ATTIVITÀ SONO:

- **Concetto di algoritmo**
- **Proprietà fondamentali degli algoritmi**
- **Pensiero computazionale**
- **Collaborazione e comunicazione**

CONSEGNA 3.1

Display the complete lyrics for the song: 99 Bottles of Beer on the Wall.

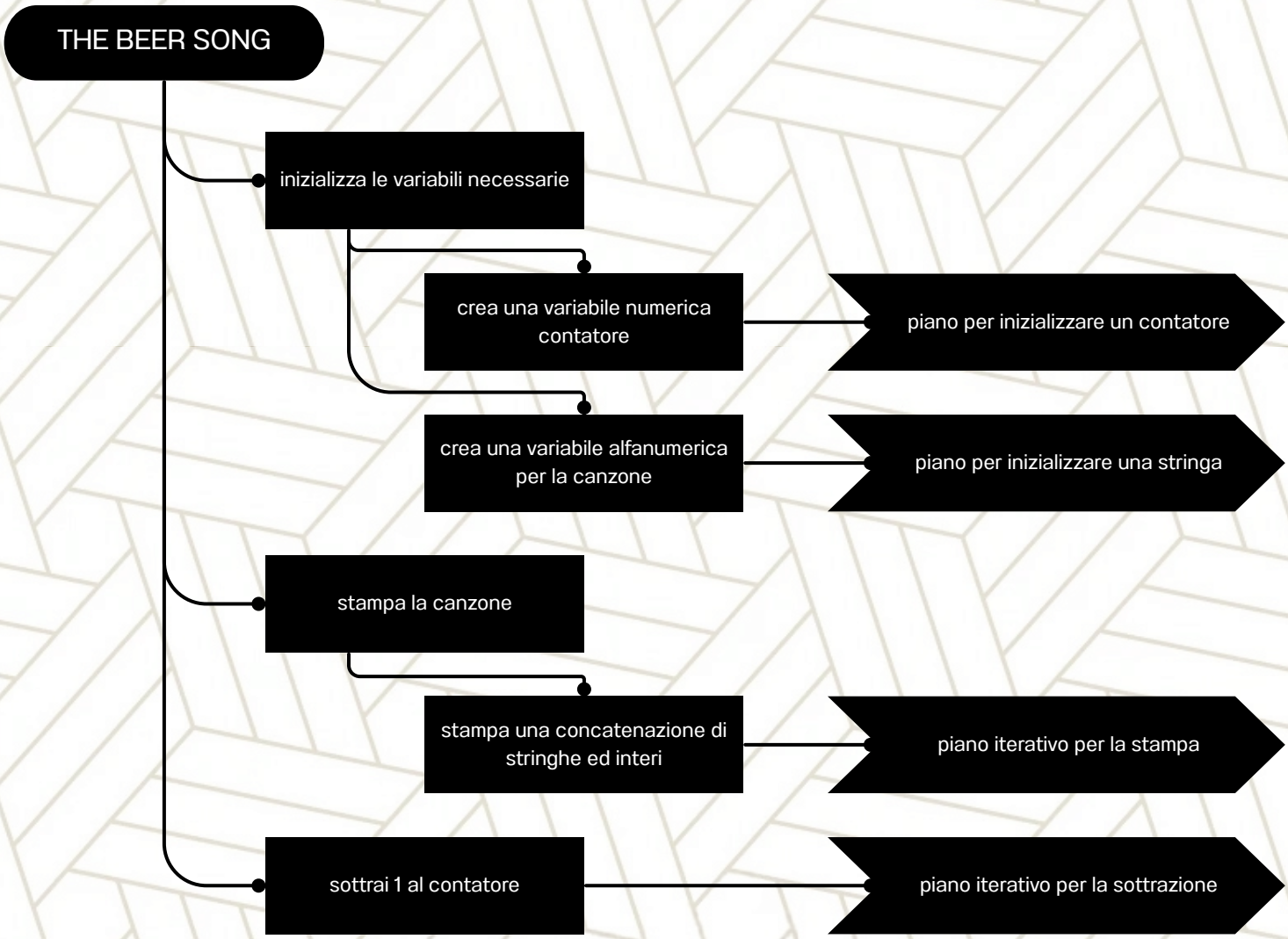
THE BEER SONG

The lyrics follow this form:
99 bottles of beer on the wall
99 bottles of beer
Take one down, pass it around
98 bottles of beer on the wall

98 bottles of beer on the wall
98 bottles of beer
Take one down, pass it around
97 bottles of beer on the wall
... and so on, until reaching 0 (zero).

Grammatical support for 1 bottle of beer is optional.

CONSEGNA 3.1



Basato su una scansione lineare semplice

Ho sempre considerato l'insegnamento come un processo complesso da affrontare per gradi: inizialmente con il supporto di un tutor e successivamente in modo autonomo. Ritengo quindi che sia fondamentale acquisire prima i concetti teorici di base, ma credo anche che alternarli con semplici esercitazioni pratiche tra un argomento e l'altro rappresenti un approccio didattico più efficace.

CONSEGNA 3.2

PROBLEMA

Hampton Court Palace (in Richmond, just southwest of London) is famous for its maze, which was planted by King William III in the 1690s. Since the palace is now open to the public as a tourist attraction, the royal authorities have decided to replace the maze with an updated version. They have estimated the number of tourists K they expect to visit during the season, and they would like the new maze to have exactly K possible routes through it, allowing each visitor to have a unique experience. The maze is a grid of N rows and M columns, where each cell can be either empty or contain a hedge. The maze is surrounded by a fence, and there is a single entrance and a single exit. The entrance is in the top-left cell, and the exit is in the bottom-right cell. The visitor must complete the maze by making only right and down moves. Due to space constraints, the maze can have no more than 200 rows and no more than 200 columns. Your task is to design a maze with exactly K routes from the entrance to the exit involving only right and down moves

I PATTERN RICHIESTI SONO:

- **Accumulation Pattern (Utilizzato per accumulare elementi che soddisfano una certa condizione)**
  - **Usato nel problema per contare il numero ed il tipo di percorsi trovati**
- **Recursion Pattern (Viene impiegato per costruire soluzioni in cui il problema viene scomposto in versioni più piccole di sé stesso)**
  - **verrà utilizzato per alleggerire la complessità della soluzione e dividerla in step**
- **Searching Pattern (Serve per esplorare soluzioni possibili in uno spazio per ottenere il numero desiderato di percorsi)**
  - **l'algoritmo centrale per la soluzione del problema**
- **Extremes Pattern**
  - **Analizzando il problema, è emerso il dubbio che il valore di K, essendo fornito dall'utente, potesse risultare non valido, ovvero non appartenente all'insieme dei valori per cui esiste almeno una configurazione del labirinto con esattamente K percorsi**

CONSEGNA 3.2

PER QUANTO RIGUARDA I PATTERN ELEMENTARI

Trattandosi di un problema articolato, nella sua implementazione intervengono tutti i principali pattern elementari, i quali si manifestano nelle seguenti situazioni specifiche:

Sequenza:
- **Inizializzazione della matrice dei percorsi**
- **Lettura delle dimensioni della griglia o del valore K**
- **Costruzione della griglia finale da stampare**

Selezione:
- **Controllo se una cella è bloccata da una siepe**
- **Verifica dei casi base in una funzione ricorsiva**
- **Validazione dell'input K (collegandoci all'ipotesi fatta in precedenza)**

Iterazione:
- **Scansione della griglia per calcolare i cammini**
- **Aggiornamento dinamico della tabella dei percorsi**
- **Eventuale esplorazione di configurazioni per trovare una che genera esattamente K percorsi**

Utilizzo di strutture dati:
- **Uso di una matrice 2D**
- **Aggiornamento dei valori della matrice**

Manipolazione ed inilizializzazione delle variabili:
- **Inizializzazione e incremento di contatori**
- **Impostazione iniziale della griglia**
- **Assegnazioni durante la costruzione della griglia**

CONSEGNA 4

PROBLEMA

Scrivere un programma che calcoli il fattoriale di un numero intero fornito dall'utente, in linguaggio C:

```
int n, fatt, i;
printf("Inserire il numero N:\n");
scanf("%d",&n);
fatt=1;
for(i=2; i<=n; i++)
  fatt=fatt*i;
printf("Fattoriale di %d = %d\n",n,fatt);
```

CONCETTI RICHIESTI

Tipi di dato

Poiché il linguaggio C adotta un sistema di tipizzazione statica e restrittiva, gli studenti devono possedere una conoscenza chiara della gestione dei tipi in C. È fondamentale comprendere quali sono i tipi primitivi disponibili (ad esempio int, float, char, double) e in quali situazioni ciascuno è più appropriato, al fine di prevenire errori di tipo, comportamenti indefiniti e perdite di precisione nei calcoli.

T1 Quali di questi è un tipo INT?

1. **3.2**
2. **7**
3. **true**
4. **"a"**
5. **"5"**

CONSEGNA 4

Dichiarazione di variabili

Nel linguaggio C, una variabile deve essere dichiarata esplicitamente prima del suo utilizzo, specificandone il tipo. La dichiarazione stabilisce il nome, il tipo e la visibilità della variabile all'interno del blocco di codice. È importante notare che non è consentita la dichiarazione multipla della stessa variabile all'interno dello stesso ambito (scope), poiché ciò genera un errore di compilazione. Tuttavia, una variabile può essere assegnata più volte, ovvero il suo valore può essere modificato nel corso dell'esecuzione del programma.

DV1 Può il valore di una variabile cambiare durante l'esecuzione?

DV2 Il seguente codice è valido?

```
int x;
int y;
int y = 10;
```

Assegnazione

Nell'assegnazione di una variabile l'operatore di assegnazione non è commutativo, i tipi delle due espressioni devono essere compatibili e per i tipi primitivi, l'assegnazione comporta una copia del valore

A1 Di che tipo deve essere la variabile x affinché il codice x = 1; funzioni correttamente?

A2 Qual è il valore della variabile x dopo l'esecuzione del codice x = 1;?

A3 Le due istruzioni x = 1; e 1 = x; sono equivalenti?

A4 Qual è il valore di a dopo l'esecuzione del seguente frammento di codice?

```
a = 1;
b=a+2;
b = 3;
```

CONSEGNA 4

Inizializzazione

L’inizializzazione rappresenta una costruzione composta, in cui si combinano due operazioni fondamentali: la dichiarazione di una variabile e l’assegnazione di un valore iniziale. Questo processo può essere interpretato come la fusione di entrambe le azioni in un'unica istruzione, tipicamente espressa nella forma

I1 Il segmento di codice int x = 1; è equivalente a:

```
Int x;
x = 1;
```

Sequence

Una sequenza di operazioni rappresenta l'esecuzione ordinata e lineare di istruzioni, dove ciascuna viene eseguita una dopo l’altra secondo un ordine ben definito.
Dal punto di vista sintattico, è importante sottolineare che la fine di un’istruzione non coincide con la fine della riga, ma è indicata dal punto e virgola (;), che funge da delimitatore esplicito tra i comandi.

S1 Il segmento di codice int x = 1; int y = 2; è equivalente a: int x; x = 1; int y; y = 2;

S2 Questo segmento di codice eseguirà sempre nello stesso ordine?

```
int x = 1;
int y = 3;
x = y + 4;
```

Loop For

Il costrutto for è una struttura di controllo iterativa che consente di eseguire ripetutamente un blocco di istruzioni per un numero determinato di volte. È particolarmente utile quando il numero di iterazioni è noto a priori.

LF1 Il ciclo for esegue prima l'aggiornamento o la condizione di controllo?

LF2 Il ciclo seguente quante volte verrà eseguito?

```
for (int i = 1; i <= 5; i++) {
   printf("%d\n", i);
}
```

CONSEGNA 4

Condizione

Una struttura condizionale può prevedere o meno un ramo else; tuttavia, indipendentemente dalla sua presenza esplicita, è fondamentale che la logica del programma consideri entrambe le possibilità previste dalla condizione.

C1 Qual è il valore della variabile x dopo l'esecuzione di questo segmento di codice?

```
int x = 1;
if(x > 2)
  x = 2
```

C2 Qual è il valore della variabile x dopo l'esecuzione di questo segmento di codice?

```
int x = 1;
if (x > 2)
  x = 2;
else
  x = 0;
```

Block Statements

Qual è il valore assunto dalla variabile x al termine dell'esecuzione dei seguenti due segmenti di codice?

B1

```
int x = 1;
if (x > 2) {
x = 2;
x = x + 5;
}
```

B2

```
int x = 1;
if (x > 2)
x = 2;
x = x + 5
```

CONSEGNA 4

Input/output

Input: è il processo attraverso cui un programma riceve dati da una fonte esterna (come tastiera, file o rete). Deve essere coerente con il tipo e il formato attesi, altrimenti può generare errori o comportamenti imprevisti. L'input è solitamente asincrono e controllato dal flusso del programma.
Output: è il processo con cui un programma restituisce informazioni all'esterno (ad esempio su schermo, file o rete). Deve essere chiaro, formattato correttamente e coerente con le aspettative dell'utente o del sistema destinatario. L'output è deterministico: dipende direttamente dai dati elaborati e dalla logica implementata.

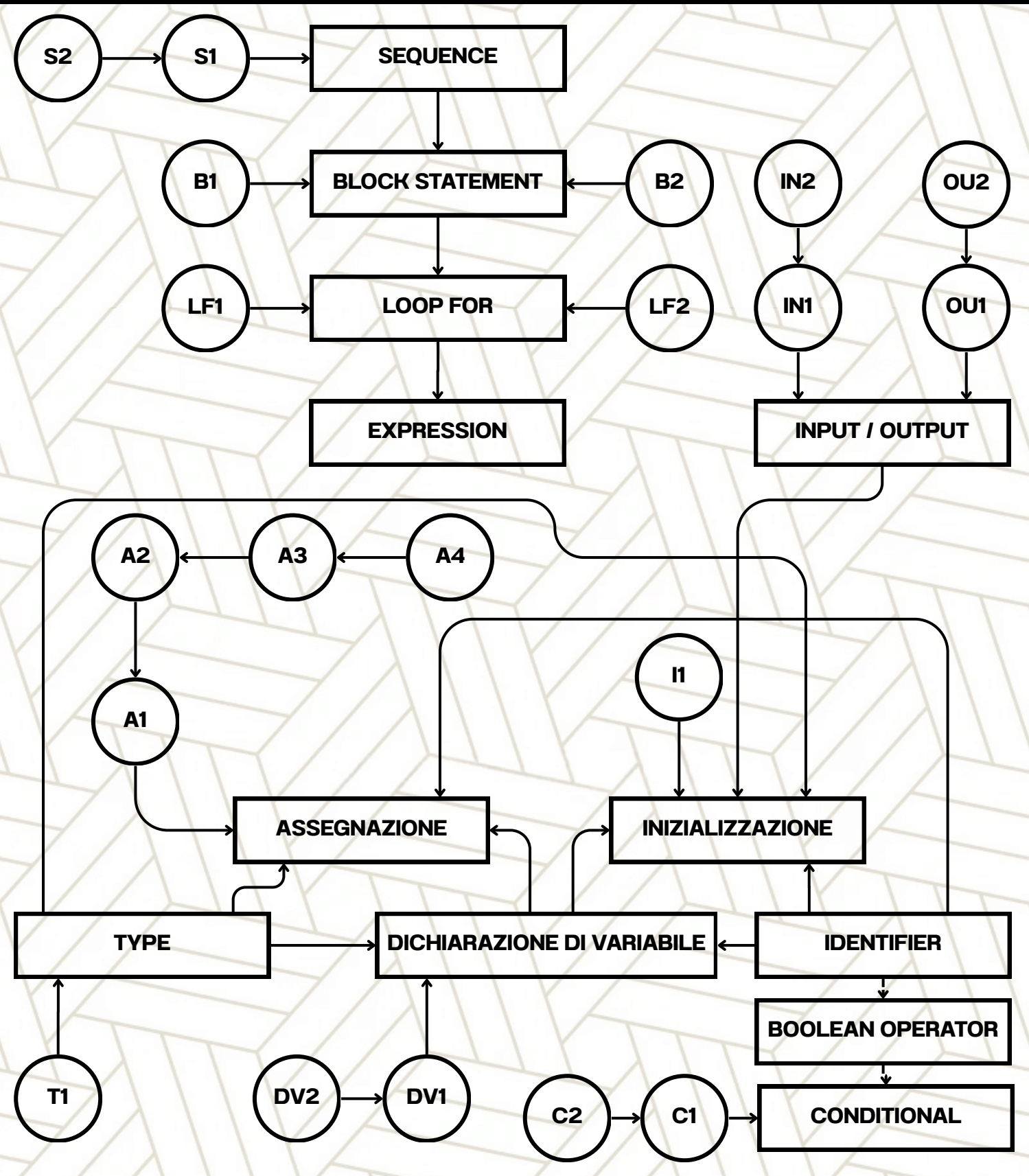
IN L'input avviene prima dell'elaborazione dei dati?

IN Il tipo di dato inserito come input deve corrispondere al tipo atteso?

OU È possibile usare più comandi di output in un programma?

OU L'output serve a mostrare i risultati del programma?

CONSEGNA 4


S2
S1
SEQUENCE
B1
BLOCK STATEMENT
B2
IN2
OU2
LF1
LOOP FOR
LF2
IN1
OU1
EXPRESSION
INPUT / OUTPUT
A2
A3
A4
A1
I1
ASSEGNAZIONE
INIZIALIZZAZIONE
TYPE
DICHIARAZIONE DI VARIABILE
IDENTIFIER
BOOLEAN OPERATOR
T1
DV2
DV1
C2
C1
CONDITIONAL

CONSEGNA 4

POSSIBILI MISCONCEPTION

- **L'inizializzazione di una variabile ne fissa definitivamente il valore, impedendo qualsiasi successiva modifica, in quanto si ritiene che il sistema mantenga e protegga quel valore iniziale**

- **l’input ha priorità sull’assegnazione e sovrascrive automaticamente il valore della variabile**

- **Si crede erroneamente che una variabile possa contenere un valore solo temporaneamente: appena viene usata in un'operazione, il suo contenuto sparisce e viene subito rimpiazzato dal valore successivo fornito in input**

CONSEGNA 5

ATTIVITÁ WIKIPASTA

Fase 1

Consegna:

A ciascuna coppia di alunni vengono forniti un foglio con una frase in grandi caratteri e un foglietto con la stessa frase già formattata. Inoltre, sono messi a disposizione diversi materiali (pasta, legumi, carta colorata, perline, ecc.) e strumenti di supporto come carta e penna.

Svolgimento:

Gli alunni devono decorare il testo non formattato usando gli oggetti a disposizione, con l'obiettivo di riprodurre visivamente la formattazione del testo guida. Durante l'attività, gli oggetti vengono utilizzati in modo analogico: ad esempio, spaghetti per sottolineare, pastina gialla per evidenziare, perline per indicare il cambio di colore. Non mancano usi creativi e personali degli oggetti.

Conclusione:

L'attività stimola una riflessione sul concetto di formattazione testuale, attraverso un approccio concreto e visivo. Gli alunni sviluppano una comprensione intuitiva delle funzioni grafiche e comunicative della formattazione.

Fase 2

Consegna:

In una seconda fase dell'attività, viene introdotto un sistema di costi associato agli oggetti decorativi. Alle coppie viene lanciata una sfida: riprodurre la formattazione del testo spendendo il meno possibile. I costi sono calibrati per rendere più costosi gli oggetti più intuitivamente utili (es. spaghetti o perline rosse), incentivando così l'uso di oggetti più economici (pastina, lenticchie).

Svolgimento:

Gli alunni iniziano a usare gli oggetti in modo più simbolico, abbandonando l'approccio analogico iniziale. Attribuiscono significati convenzionali agli oggetti e inventano strategie di codifica, ad esempio usando delimitatori per indicare l'inizio e la fine della formattazione. Si sviluppano soluzioni sempre più ingegnose ma anche potenzialmente ambigue, stimolando riflessioni sulla necessità di regole condivise e non ambigue. La competizione viene alimentata aggiornando la classe sui progressi, stimolando l'emulazione tra compagni che favorisce l'apprendimento, purché non degeneri.

Conclusione:

La discussione finale evidenzia i vantaggi dell'uso di marcatori e la necessità di definire regole chiare. Gli alunni comprendono che più soluzioni possono essere corrette, purché supportate da convenzioni condivise. L'esperienza viene infine collegata a contesti reali, come il formato di modifica testuale usato in Wikipedia, rendendo evidente il legame tra attività didattica e tecnologie digitali.

CONSEGNA 5

Fase 3

Consegna:

Nella fase conclusiva di Wikipasta, agli alunni viene chiesto di formattare un testo utilizzando un'applicazione digitale progettata ad hoc, che adotta regole simili a quelle del linguaggio Wiki. Le regole di formattazione sono predefinite dal software e si basano sull'uso di simboli speciali come delimitatori, mostrati in modo schematico a lato dello schermo.

Svolgimento:

Gli alunni applicano le regole formali senza particolari difficoltà, avendo ormai acquisito l'idea che ogni convenzione ha valore purché condivisa e utile a "far capire" al sistema cosa fare. Il software fornisce un feedback immediato che permette agli alunni di vedere in tempo reale l'effetto delle loro azioni, rinforzando il legame tra istruzione e risultato visivo.

Conclusione:

Questa attività finale rafforza la comprensione delle convenzioni simboliche nella formattazione del testo e prepara all'introduzione di linguaggi più complessi come l'HTML. La transizione ai tag HTML viene così facilitata, grazie alla familiarità acquisita con i delimitatori e all'effetto motivante della visualizzazione diretta dei risultati.

OBIETTIVI DA RAGGIUNGERE AL TERMINE DELL'ATTIVITÁ

- **riconoscere la differenza tra dati e metadati in alcuni ambiti semplici (es. HTML, semplici linguaggi di descrizione dei dati, ...).**
- **valutare vantaggi e svantaggi di rappresentazioni alternative della stessa informazione;**
- **realizzare esperienze di raccolta ed analisi dati attraverso sensori e di controllo di dispositivi esterni;**
- **scrivere semplici programmi in un linguaggio testuale di programmazione rispettandone la sintassi**

CONSEGNA 5

TEMI INFORMATICI AFFRONTATI NELL'ATTIVITÁ WIKIPASTA

- **Codifica e rappresentazione delle informazioni**
- **Formalizzazione e convenzioni**
- **Efficienza e ottimizzazione**
- **Ambiguità**
- **Debugging**